

Program 4 – Documentation

Title: Integrate Git with Docker for Source-Controlled Application Builds

Introduction:

Modern software development heavily relies on version control, containerization, and continuous deployment workflows.

Git enables teams to track source-code changes, while Docker provides an isolated environment to build, package, and run applications consistently across machines.

Integrating Git with Docker ensures that every build is reproducible, source-controlled, and automated. This combination is widely used in CI/CD pipelines, DevOps workflows, and cloud deployments.

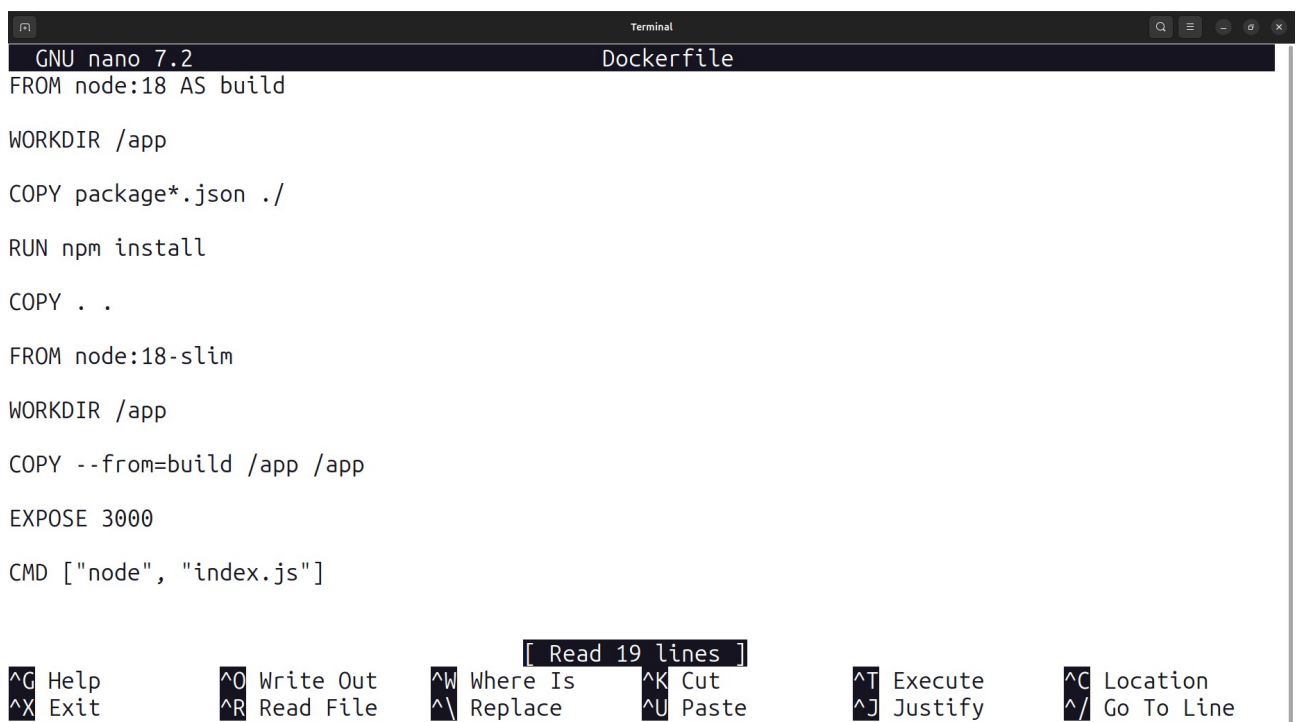
This documentation explains how to integrate Git with Docker for source-controlled application builds, using screenshots similar to your own workflow.

Project Structure:

Program4/

- | — index.js
- | — package.json
- | — package-lock.json
- | — Dockerfile
- | — node_modules/

Write the Docker file



```
Terminal
GNU nano 7.2 Dockerfile
FROM node:18 AS build

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

FROM node:18-slim

WORKDIR /app

COPY --from=build /app /app

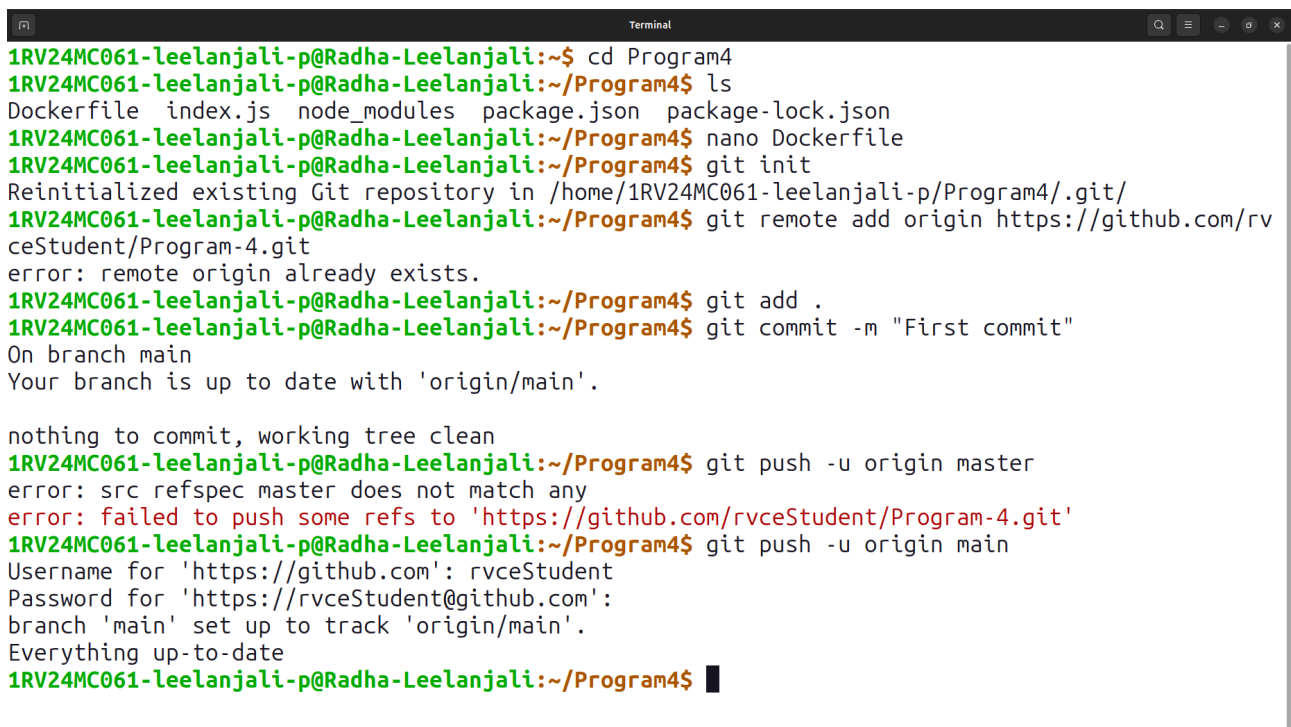
EXPOSE 3000

CMD ["node", "index.js"]

[ Read 19 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```

This screenshot shows the contents of your `Dockerfile` inside the `nano` text editor. This file tells Docker how to package your application.

- **Multi-Stage Build:** You are using an advanced technique called a "multi-stage build."
 - **Stage 1 (FROM node:18 AS build):** You use a full Node.js image to copy your files and run `npm install` to get all dependencies.
 - **Stage 2 (FROM node:18-slim):** You switch to a "slim" (smaller) version of Node.js for the final product. You copy the files from the build stage (`COPY --from=build . .`). This makes your final application much lighter and faster to download.
- **Configuration:** You are exposing port **3000** and telling the container to start by running `node index.js`.

A terminal window titled "Terminal" showing a series of commands and their outputs. The user is in a directory named "Program4". They list files, initialize git, add a remote origin, add files, commit, and attempt to push to GitHub. The push to the master branch fails with an error, but the push to the main branch succeeds after a username and password are provided.

```
1RV24MC061-leelanjali-p@Radha-Leelanjali:~$ cd Program4
1RV24MC061-leelanjali-p@Radha-Leelanjali:~/Program4$ ls
Dockerfile index.js node_modules package.json package-lock.json
1RV24MC061-leelanjali-p@Radha-Leelanjali:~/Program4$ nano Dockerfile
1RV24MC061-leelanjali-p@Radha-Leelanjali:~/Program4$ git init
Reinitialized existing Git repository in /home/1RV24MC061-leelanjali-p/Program4/.git/
1RV24MC061-leelanjali-p@Radha-Leelanjali:~/Program4$ git remote add origin https://github.com/rvceStudent/Program-4.git
error: remote origin already exists.
1RV24MC061-leelanjali-p@Radha-Leelanjali:~/Program4$ git add .
1RV24MC061-leelanjali-p@Radha-Leelanjali:~/Program4$ git commit -m "First commit"
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
1RV24MC061-leelanjali-p@Radha-Leelanjali:~/Program4$ git push -u origin master
error: src refspec master does not match any
error: failed to push some refs to 'https://github.com/rvceStudent/Program-4.git'
1RV24MC061-leelanjali-p@Radha-Leelanjali:~/Program4$ git push -u origin main
Username for 'https://github.com': rvceStudent
Password for 'https://rvceStudent@github.com':
branch 'main' set up to track 'origin/main'.
Everything up-to-date
1RV24MC061-leelanjali-p@Radha-Leelanjali:~/Program4$
```

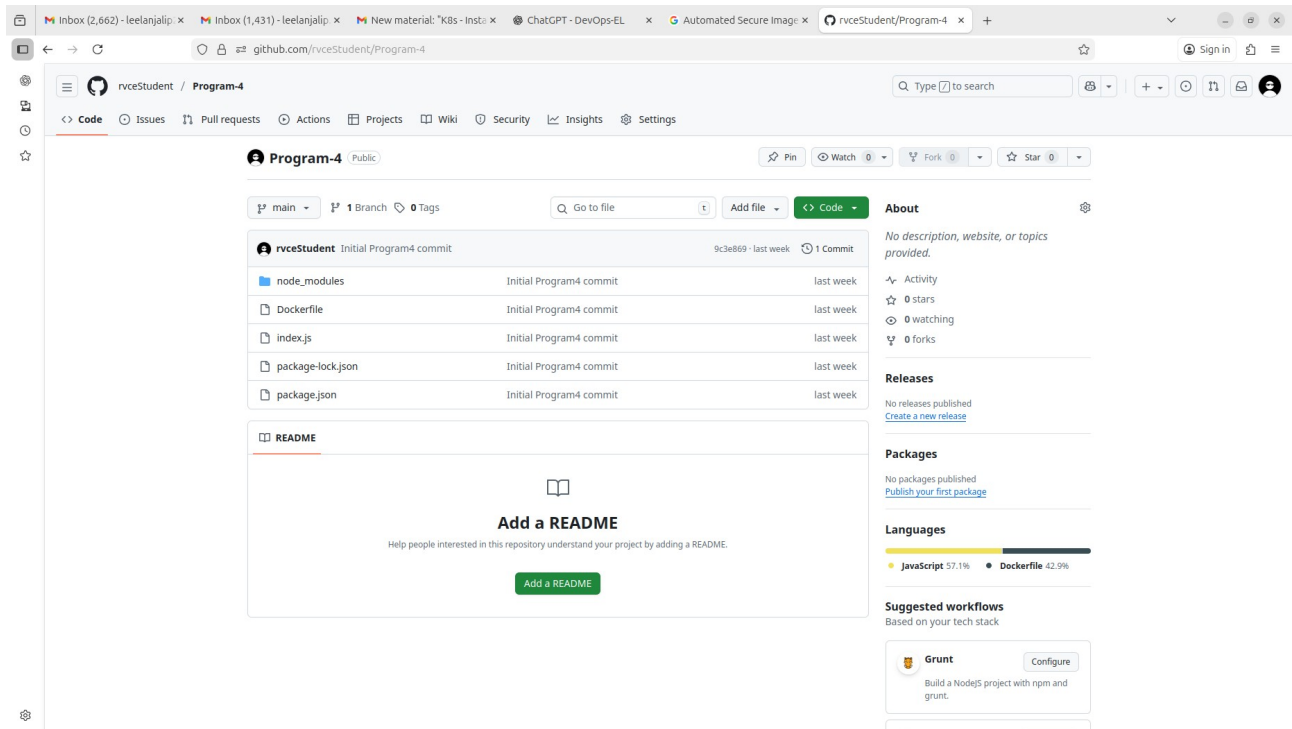
This screenshot captures the "struggle" and eventual success of pushing your code to GitHub. Here is a play-by-play of the commands entered:

1. `git remote add origin . . .`: You tried to link your local folder to GitHub.
 - **Result:** Error. The terminal says `error: remote origin already exists`. This means you had already linked this folder to GitHub previously.
2. `git commit -m "First commit"`: You tried to save your changes.
 - **Result:** "nothing to commit, working tree clean". This means all your files were already saved/committed before you ran this command.
3. `git push -u origin master`: You tried to upload your code to the `master` branch.

- **Result:** Error. `src refspec master does not match any`. This happened because GitHub changed the default branch name from `master` to `main` a few years ago. Your local computer uses `main`, but you tried to push to `master`.

4. `git push -u origin main`: You corrected the branch name to `main`.

- **Result:** Success. The terminal asked for your username and password, authenticated you, and then reported **Everything up-to-date**.



This screenshot shows your remote repository on GitHub named `rvceStudent/Program-4`.

- **Files:** You have successfully uploaded a standard Node.js project structure:
 - `index.js`: The main application logic.
 - `package.json` & `package-lock.json`: Dependency management files.
 - `Dockerfile`: The instructions to build the container.
 - `node_modules`: The installed libraries (Note: Usually, `node_modules` is included in a `.gitignore` file and not uploaded to GitHub to save space, but it is present here).
- **Status:** The commit message "Initial Program4 commit" indicates this is a fresh project that has just been set up.